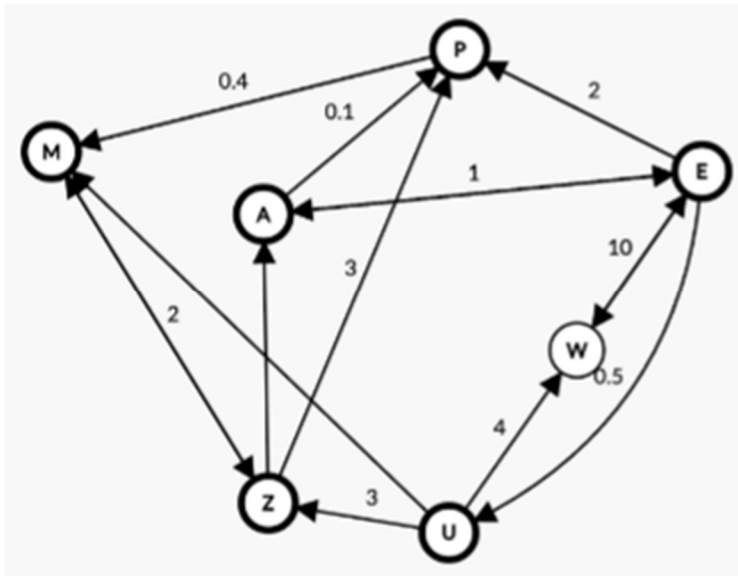


Assignment Redo:

1. Dijkstra:



Z = 0

All other vertices = infinity

Outgoing from Z:

- Z → A = 0
- Z → P = 3
- Z → M = 2

Outgoing from A:

- A → P = 0.1
- A → E = 1

Outgoing from P:

- P → M = 0.5

Outgoing from M:

- M → Z = 2.5 > Z = 0
- Don't update Z

Back to P:

No more edges

Back to A:

Outgoing from A:

- A → E = 1

Outgoing from E:

- E → U = 1.5
- E → A = 2
- E → W = 10

Outgoing from U:

- U → Z = 4.5
- U → M = 1.5 < M = 2
- Update M to 1.5.

Outgoing from U:

- U → Z = 4.5 > Z = 0
- Don't update Z

Back to E

Outgoing from E:

- E → A = 2 > A = 0
- E → W = 10
- Don't update A
- E → W = 11

Outgoing from W:

- W → E = 21 > E = 11
- Don't update E

Back to E.

Back to A.

Back to Z.

Outgoing from Z:

- Z → P = 3 > P = 0.1
- Z → M = 2 > M = 0.5

Don't update P or M

$Z = 0 [Z]$
 $A = 0 [Z, A]$
 $P = 0.1 [Z, A, P]$
 $E = 1 [Z, A, E]$
 $U = 1.5 [Z, A, E, U]$
 $M = 1.5 [Z, A, E, U, M]$
 $W = 11 [Z, A, E, W]$

5. Algorithm Analysis:

Unweighted shortest path:

The time complexity is $O(V^2)$ where V is the number of vertices represented in the matrix. There are nested loops, a for loop within a while loop, that are responsible for doing the search. The for loop takes the visited vertex (row) and iterates V times, checking if each vertex is adjacent with each iteration before pushing the adjacent ones to a queue. The while polls the starting and adjacent vertices at the start of each iteration. In the worst case scenario, the queue will at some point store all vertices in the graph, making the outer loop iterate V times. There is one additional standalone for loop that use constant statements to populate default values for a pred array of size V , which gives it an $O(V)$ time complexity. There is also a function to get the final shortest path. Starting from the target, it adds the vertex to a path list before pointing to its predecessor and repeating until the start is added. In the worst case scenario, the path takes all vertices, making it $O(V)$. With all of this, the time complexity of this solution is $(O(V) + O(V) + O(V^2) + O(V)) = O(V^2)$.

The space complexity is $O(V)$. There are 4 variables that use space relative to the amount of vertices represented in the input: visited, pred, q, and path. The visited and pred arrays always store V elements, as each index stores info for each vertex. The path list stores V elements at most if it takes all vertices to get from point to point. In the worst case scenario for the q queue, one of the vertices is adjacent to all other vertices, making the queue store almost the whole graph. Other variables like l, node, and current, are integer variables that store only constant space. With all this in mind, the space complexity is $O(V)$.